Project 10: Code Reuse / Refactoring with AI

Key papers: links and summaries

Evgenii Lialin Roman Vikhman

Venue status under the course rule (A/A+/A* IR/NLP/CL venues, 2025 onward): **main venue** counts; **Findings / Industry** needs the instructor's approval; **preprint** arXiv, not peer-reviewed at a qualifying venue. Summaries follow each paper's published abstract.

1. The problem: models do not reuse existing code

RepoExec

Findings

On the Impacts of Contexts on Repository-Level Code Generation
Hai et al. — Findings of NAACL 2025
https://aclanthology.org/2025.findings-naacl.82/

A benchmark for repository-level code generation that checks executability, functional correctness and *dependency utilisation*. Evaluating 18 models, it finds that full dependency context works best and that small contexts can be misleading. Pretrained models are often correct but *re-implement* dependencies; instruction-tuned models use dependencies more but sometimes add needless complexity. It introduces the **Dependency Invocation Rate (DIR)**, a metric of how often the generated code calls the available dependencies, and an instruction-tuning dataset that raises DIR by over 10%.

Relevance to our project: The closest prior work. We adopt its central idea, measuring whether the model *calls* the existing function, and move it from generation to refactoring, where the duplicated logic is already in the code and has to be recognised.

More Code, Less Reuse

preprint

More Code, Less Reuse: Investigating Code Quality and Reviewer Sentiment towards AI-generated Pull Requests
Huang et al. — arXiv:2601.21276, 2026
https://arxiv.org/abs/2601.21276

An empirical study of pull requests written by LLM agents versus humans, looking beyond pass rates at code-quality metrics and reviewers' reactions. LLM agents frequently **disregard code reuse opportunities**, producing more redundancy than human developers, yet reviewers respond to AI pull requests neutrally or even more positively. The authors argue that plausible-looking AI code hides redundancy, so technical debt accumulates silently.

Relevance to our project: Our main motivation: the lack of reuse is observed in real development and goes unnoticed in review. Our benchmark asks where in the pipeline that reuse is lost.

DependEval

Findings

DependEval: Benchmarking LLMs for Repository Dependency Understanding
Du et al. — Findings of ACL 2025
https://aclanthology.org/2025.findings-acl.373/

A hierarchical benchmark built from 2,683 real repositories in 8 programming languages, with three tasks: dependency recognition, repository construction and multi-file editing. Over 25 LLMs show substantial gaps in repository-level understanding.

Relevance to our project: Shows that understanding repository dependencies is a known weakness of LLMs; reuse requires exactly this understanding.

2. Refactoring benchmarks

RefactorBench

ICLR 2025

RefactorBench: Evaluating Stateful Reasoning in Language Agents Through Code
Gautam et al. — ICLR 2025
https://arxiv.org/abs/2503.07832

100 handcrafted multi-file refactoring tasks in popular open-source (Python) repositories, each given by three natural-language instructions of different specificity. Agents solve only 22% with the base instructions, against 87% for a human developer under time pressure. Trajectory analysis finds that agents lose track of past actions; conditioning the agent on a representation of its state improves results by 43.9%.

Relevance to our project: Python refactoring benchmark; it measures whether the task is completed, not whether existing code is reused.

CodeTaste

ICML 2026

CodeTaste: Can LLMs Generate Human-Level Code Refactorings?
Thillen et al. — ICML 2026
https://arxiv.org/abs/2603.04177

Starts from the observation that coding agents accumulate complexity, **duplication** and architectural debt. A benchmark mined from large multi-file open-source refactorings, scored with test suites plus static checks (dataflow-based) that verify removal of undesired and introduction of desired code patterns. Agents implement refactorings well when they are specified in detail but often fail to *discover* the refactoring a human chose; a propose-then-implement decomposition helps.

Relevance to our project: Closest in spirit: the gap is between executing a refactoring and choosing it. Our finding that models ignore a helper already in context is a specific case of this "choice" failure.

SWE-Refactor

preprint

SWE-Refactor: A Repository-Level Benchmark for Real-World LLM-Based Code Refactoring
Xu et al. — arXiv:2602.03712, 2026
https://arxiv.org/abs/2602.03712

1,099 developer-written, behaviour-preserving refactorings mined from 18 Java projects (922 atomic, 177 compound), each validated by compilation, tests and refactoring-detection tools. Nine LLMs are evaluated; compound refactorings are the main source of failure (a Codex agent reaches 39.4%).

Relevance to our project: Our task validation follows the same principle (behaviour preserved, checked by tests), but we target Python and reuse specifically.

MANTRA

preprint

MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration
Xu et al. — arXiv:2503.14340, 2025
https://arxiv.org/abs/2503.14340

An agent framework for method-level refactoring that combines context-aware retrieval-augmented generation, multi-agent collaboration and verbal reinforcement learning. On 703 pure refactorings from 10 Java projects it produces code that compiles and passes tests in 82.8% of cases (8.7% for a plain LLM baseline); developers judge its refactorings as readable and reusable as human ones.

Relevance to our project: Shows that retrieval helps refactoring; we test whether retrieval is in fact the bottleneck for reuse (it mostly is not).

3. Retrieval of repository code

CodeRAG

EMNLP 2025

CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented Repository-Level Code Completion
Zhang et al. — EMNLP 2025
https://aclanthology.org/2025.emnlp-main.1187/

Addresses poor query construction, single-path retrieval and the mismatch between retriever and code LLM in repository-level completion, using log-probability-guided queries, multi-path retrieval and preference-aligned reranking (BestFit). Outperforms prior methods on ReccEval and CCEval.

Relevance to our project: Representative of work that improves *retrieval*. Our funnel shows that, for reuse, the model's decision costs more than retrieval misses.

CoRet

ACL 2025

CoRet: Improved Retriever for Code Editing
Fehr et al. — ACL 2025 (short)
https://aclanthology.org/2025.acl-short.62/

A dense retriever for code editing that combines code semantics, repository structure and call-graph dependencies, trained with a loss designed for repository-level retrieval. Improves retrieval recall by at least 15 points on SWE-bench and Long Code Arena bug localisation.

Relevance to our project: Retrieval for *editing*, our setting; a candidate for the dense-retrieval condition we have not run yet.

ReflectCode

Findings

Beyond Function-Level Search: Repository-Aware Dual-Encoder Code Retrieval with Adversarial Verification
Liu et al. — Findings of EMNLP 2025
https://aclanthology.org/2025.findings-emnlp.1147/

Introduces RepoAlignBench for repository-level retrieval driven by change requests, and ReflectCode, a dual-encoder (code and documentation towers) that uses an LLM to integrate syntax, function dependencies and the intent of the change. Improves top-5 accuracy by 12.2% and recall by 7.1%.

Relevance to our project: Retrieval driven by a change request, close to our refactoring request.

Think-Search-Patch

Industry

Think-Search-Patch: A Retrieval-Augmented Reasoning Framework for Repository-Level Code Repair
Xiong et al. — EMNLP 2025 Industry Track
https://aclanthology.org/2025.emnlp-industry.109/

A three-stage framework for repository-level repair: break the task down and write a search query, retrieve

snippets (e.g. with E5), then generate a patch; trained with SFT and DPO. Improves retrieval accuracy and repair success on SWE-bench Lite, including multi-file bugs.

Relevance to our project: Another retrieve-then-edit pipeline, which assumes the model uses what is retrieved; our results question that assumption for reuse.

4. How models use context and instructions

Lost in the Middle

TACL 2024

Lost in the Middle: How Language Models Use Long Contexts
Liu et al. — TACL 2024
https://aclanthology.org/2024.tacl-1.9/

On multi-document question answering and key-value retrieval, performance depends strongly on the position of the relevant information: it is highest at the beginning or end of the context and drops in the middle, even for long-context models.

Relevance to our project: Basis of our position ablation: we place the helper first or last among the retrieved snippets. Three of four models change their reuse rate with the position.

CodeIF

Industry

CodeIF: Benchmarking the Instruction-Following Capabilities of Large Language Models for Code Generation
Yan et al. — ACL 2025 Industry Track
https://aclanthology.org/2025.acl-industry.89/

The first benchmark of how well LLMs follow task-oriented instructions in code generation, covering function synthesis, debugging, algorithmic refactoring and code explanation across difficulty levels and domains; it highlights gaps in producing consistent, maintainable code.

Relevance to our project: Motivates our two instructions: a generic refactoring instruction versus an explicit "reuse existing functions" instruction, which raises reuse in every condition.